

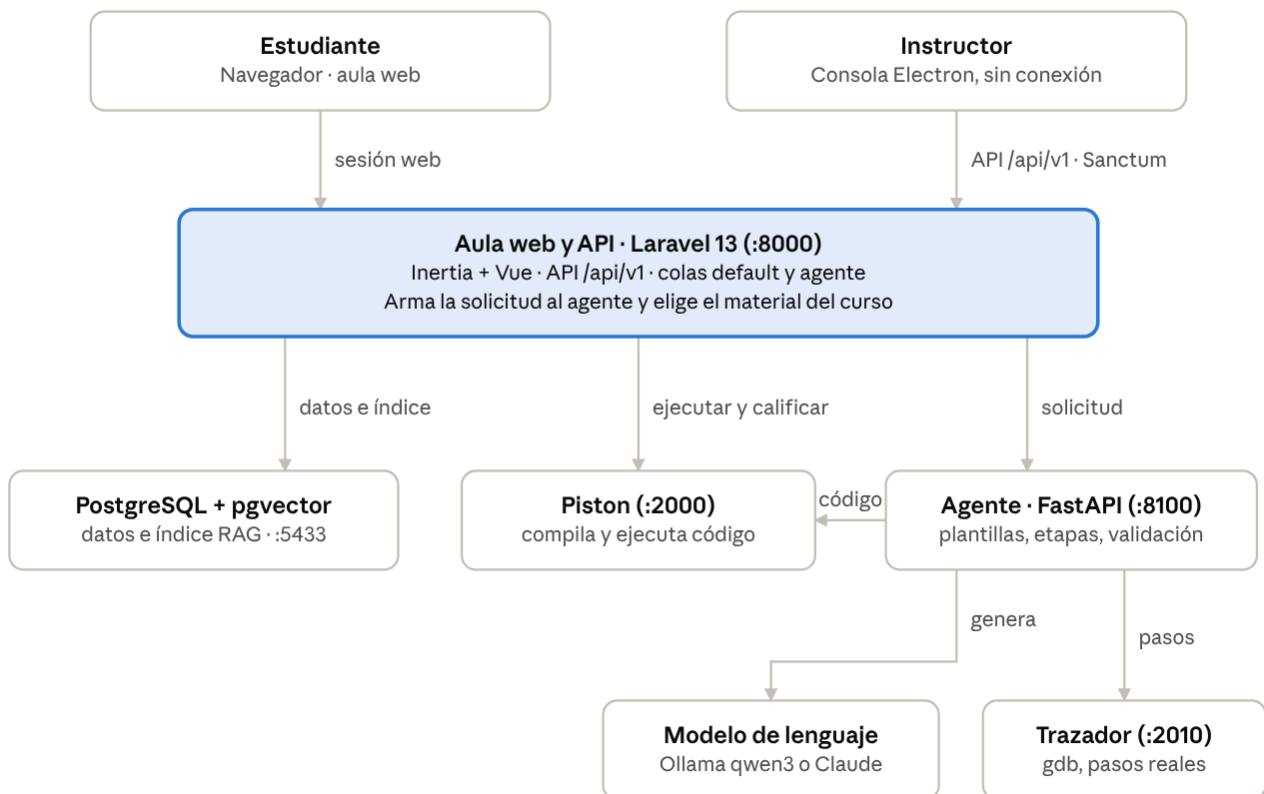
Manual técnico de CLT4BP

Oct 7, 2026 · @Carlos Sandoval

CLT4BP es una plataforma híbrida para enseñar programación con el modelo instruccional CLT4BP (TCC, 4C/ID, ARCS, instrucción diferenciada y protocolos verbales): un aula web para estudiantes, una consola de escritorio para el instructor y un agente de IA local que propone material que solo se publica con la aprobación del instructor. Este manual describe la versión v0.7.0 más la rama del agente local (prompt `sistema-v12`).

Arquitectura

Laravel es el único que habla con el agente; solo el agente llama al trazador



arquitectura · quién llama a quién

El estudiante y el instructor solo hablan con Laravel. Laravel guarda los datos, ejecuta el código de los estudiantes en Piston y es el único que llama al agente, con un token compartido y solo seudónimos y datos agregados. El agente usa el modelo de lenguaje (Ollama local o Claude), Piston para calcular salidas y el trazador para los pasos de las trazas. Mailpit, el correo de desarrollo, queda fuera del dibujo.

Componentes y tecnologías

El sistema son tres aplicaciones (aula, consola, agente) y cuatro servicios de apoyo en contenedores; en desarrollo todo escucha solo en 127.0.0.1 .

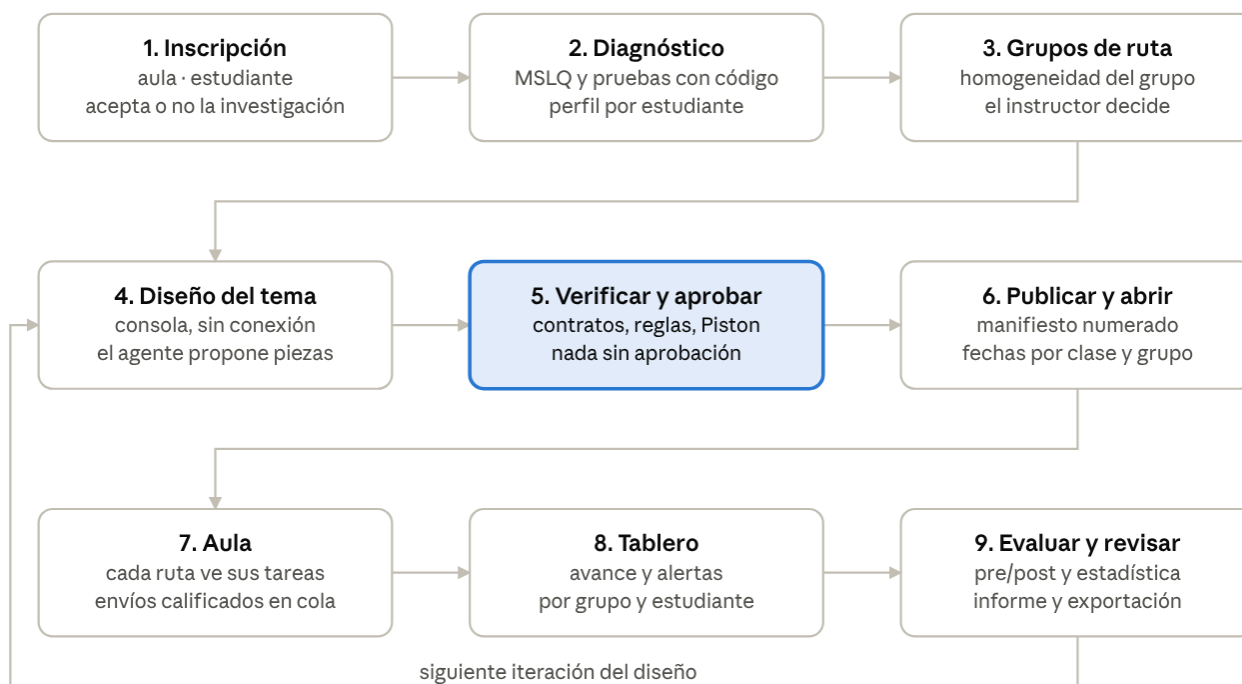
Componente	Ubicación	Tecnología	Puerto local	Responsabilidad
Aula web y API	apps/web	Laravel 13 (PHP 8.3), Inertia 3, Vue 3 + TypeScript, Tailwind 4, Sanctum, Fortify, spatie/laravel-permission, opis/json-schema	8000	Aula del estudiante, diagnóstico y perfil, publicación, API /api/v1 para la consola, colas, índice del material (RAG)
Consola del instructor	apps/desktop	Electron 44 + electron-vite 5, Vue 3, Pinia, SQLite local (node:sqlite), electron-updater	—	Diseño instruccional sin conexión con sincronización, asistente del agente, publicación, análisis y exportación
Agente de IA	services/agent	FastAPI (Python 3.13), pydantic, httpx, SDK de Anthropic, pypdf	8100	Propuestas de diseño con plantillas y etapas deterministas; nunca accede a la base de estudiantes
Modelo local	Ollama (nativo)	qwen3:4b (normal), qwen3:8b (calidad alta), bge-m3 (embeddings)	11434	Generación de texto y embeddings sin salir del equipo

Componente	Ubicación	Tecnología	Puerto local	Responsabilidad
Base de datos	contenedor <code>clt4bp-db</code>	PostgreSQL 17 + pgvector	5433	Datos de la plataforma e índice vectorial del material
Ejecución de código	contenedor <code>clt4bp-piston</code>	Piston (sandbox isolate)	2000	Compila y ejecuta C, C++ y Python: casos de prueba y salidas
Trazador	<code>services/trazador</code>	Debian con gcc/g++, gdb y Python; sin red ni privilegios, 768 MB, 128 procesos	2010	Pasos reales de un programa, línea por línea, con la memoria
Correo (desarrollo)	contenedor <code>clt4bp-mail</code>	Mailpit	1025 (SMTP), 8025 (web)	Avisos y verificación de correo
Contratos	<code>packages/contracts</code>	JSON Schema → TypeScript y pydantic generados	—	Fuente de verdad del contenido 4C/ID

Solo Laravel llama al agente, y solo el agente llama al trazador; la consola habla únicamente con la API de Laravel.

Flujo del sistema

Nada llega al aula sin pasar por la aprobación del instructor



flujo del sistema · 9 pasos y la iteración

Los pasos 1, 2 y 7 ocurren en el aula; los demás, en la consola. El agente interviene como propuesta en el resumen de grupos, la diferenciación, el diseño del tema, la evaluación y el informe de revisión; lo propuesto entra al diseño como borrador y solo llega al aula si el instructor lo aprueba y lo publica. El ciclo es iterativo: la revisión de resultados alimenta el diseño de la siguiente clase o del siguiente curso.

Modelo de datos y contratos

El contenido instruccional es un documento JSON validado contra contratos JSON Schema; la base lo guarda como elementos versionados y lo congela en publicaciones numeradas.

Contratos (`packages/contracts`)

Los esquemas de `packages/contracts/schema` son la fuente de verdad del contenido 4C/ID. `DisenoCurso` (`diseno-curso.schema.json`) reúne el curso con sus grupos y ocho listas de elementos:

Esquema	Elemento	Campos clave
objetivo	Objetivo de desempeño	codigo (OB-1...), tipo (conocimiento, habilidad, actitud), evaluacion
clase-tareas	Clase de tareas (un tema)	objetivos , descripcion_complejidad , ficha_md (ficha de diseño, solo instructor)
tarea	Tarea de aprendizaje	nivel_apoyo (ejemplo resuelto, por completar, convencional, solución libre), enunciado_md , codigo_inicial , solucion , casos_prueba , arcs , rutas , pide_autoexplicacion , colaborativa
info-soporte	Información de soporte	tipo (modelo mental, SAP, explicación, mapa conceptual, guion de video), cuerpo_md
info-procedimental	Ayuda justo a tiempo	tipo (ficha de sintaxis, guía de preguntas, errores frecuentes, ejemplo isomórfico, protocolo verbal); cuelga de una tarea o de una clase
practica-parcial	Práctica de tareas parciales	habilidad , ejercicios
variante	Diferenciación por grupo	elemento_uid , grupo_clave , cambios , diferenciacion
medio	Imagen, audio, video o protocolo grabado	segmentos , transcripcion , duracion_s

Todo elemento de diseño lleva un bloque `diseño` (paso CLT4BP, componente, efectos de la TCC aplicados, interactividad, tiempo estimado, justificación); el estudiante nunca lo ve.

Tras cambiar un esquema se regeneran los tipos y nunca se editan a mano los archivos marcados GENERADO:

1. `npm run gen` (en `packages/contracts`): tipos TypeScript y copias de los esquemas para el aula (`resources/contracts` , `resources/js/contracts`) y la consola (`src/shared/contracts.ts` , `src/main/esquemas`).
2. `npm run gen:py`: modelos pydantic del agente (`services/agent/app/contracts`).
3. `npm run check`: valida los ejemplos de `examples/` contra los esquemas.

Los tres lados validan con el mismo esquema: Laravel con opis/json-schema (ValidadorContratos), la consola antes de sincronizar y el agente con pydantic. Las reglas que JSON Schema expresa mal (por ejemplo, «exactamente uno de tarea_uid o clase_uid ») se validan en código.

Base de datos (PostgreSQL 17 + pgvector)

Área	Tablas	Qué guardan
Identidad y acceso	users , tablas de spatie/permission, personal_access_tokens , sessions , consents , audit_logs	Cuentas, roles (administrador, instructor, estudiante), tokens de la consola y del agente, consentimientos, bitácora de auditoría
Cursos	courses , course_instructor , enrollments	Curso (lenguaje, nivel, estado), instructores y su rol, inscripciones y su estado
Diagnóstico y perfil	instruments , instrument_administrations , instrument_responses , items , assessments , assessment_items , assessment_attempts , item_responses , student_profiles , group_analyses , diff_groups , group_memberships	MSLQ y pruebas, banco de ítems, perfil calculado, análisis de homogeneidad y grupos de ruta (G1, G2...)
Diseño instruccional	design_elements , design_element_versions , cle_effects , effect_rules , media_assets	Elementos del diseño con su historial, catálogo de 17 efectos de la TCC y sus reglas, medios
Agente	agent_jobs , agent_runs , agent_quotas	Solicitudes en cola, cada corrida (modelo, versión del prompt, tokens, costo, validaciones, decisión del instructor), cuotas
Publicación y aula	releases , activations , task_progress , task_submissions , learning_events , task_comments	Manifiestos publicados, fechas de apertura por clase y grupo, avance, envíos calificados, eventos de aprendizaje, comentarios

Área	Tablas	Qué guardan
Revisión	result_reviews	Informes de revisión de resultados (paso 10)
Material del curso	course_documents , document_fragments	PDF subidos y sus fragmentos con embedding vector(1024) e índice HNSW por coseno
Infraestructura	cache , jobs , job_batches , failed_jobs , notifications , password_reset_tokens	Cola, caché y notificaciones de Laravel

Tres reglas sostienen el diseño:

- **Elementos versionados.** design_elements tiene uid estable, estado (propuesta, borrador, aprobado, publicado, archivado), version , autor_tipo (humano o agente), agent_run_id y seq , la posición en el registro de cambios del curso que la consola usa para sincronizarse. Una baja es una lápida (eliminado_at), no un borrado.
- **Publicación inmutable.** releases.manifiesto congela el DisenoCurso aprobado con su número, huella SHA-256 y clave de idempotencia; revertir vuelve a la versión anterior.
- **Migraciones.** Nunca se modifica una migración ya aplicada en main : se crea una nueva.

Aula web (Laravel)

apps/web sirve el aula del estudiante con Inertia (páginas Vue renderizadas desde controladores) y la API /api/v1 que usa la consola; el cálculo pedagógico vive en PHP puro, separado del framework.

Capas

Carpeta	Contenido	Regla
app/Domain	Perfil (CalculadoraPerfil , Agrupador , AnalizadorGrupo), instrumentos (PuntuadorLikert), pruebas (CalificadorItems , ComparadorSalida), aula (AplicadorVariantes , Disponibilidad , ValidadorManifiesto , VistaEstudiante), diseño, evaluación, operación, seguridad	PHP puro, sin base de datos, con pruebas unitarias en tests/Unit/Domain
app/Services	Agente, Aula (ServicioPublicacion , ServicioAula), Código (Piston), Diagnóstico, Diseño (ServicioSincronizacion , ValidadorContratos , VerificadorCodigo), Evaluación	Orquestan dominio, base y servicios externos
app/Jobs	CalcularPerfil , AnalizarGrupo , CalificarIntento , CalificarEnvio , EjecutarTrabajoAgente , ProcesarDocumento	Los dos últimos van en la cola agente , para no bloquear calificaciones
app/Http/Controllers/Web	Aula, avance, diagnóstico, evaluación final, avisos, «Mis datos», invitaciones	Devuelven páginas Inertia (resources/js/pages)
app/Http/Controllers/Api/V1	Cursos, inscripciones, ítems, diagnóstico, diseño, medios, documentos, trazas, efectos, agente, publicación, activaciones, tablero, resultados, administración	JSON para la consola, con Sanctum

Carpeta	Contenido	Regla
app/Policies	CoursePolicy (viewAny , view , create , update , manageEnrollments , acceder)	Toda ruta nueva tiene política y una prueba que verifica el 403

Rutas principales

- **Estudiante** (routes/web.php , sesión web con auth , verified , no-suspendido , personal-2fa): inicio, cursos, diagnóstico (cuestionarios MSLQ y pruebas con ejecución de código), aula aula/{curso} (mapa, clase, tarea, práctica, avance; protegida por can:acceder), envíos, comentarios, eventos, evaluación final, avisos y «Mis datos».
- **Consola** (routes/api.php , prefijo v1 , auth:sanctum + habilidad consola): courses , enrollments , items y assessments , diagnostico y grupos , design (lectura, sync , versiones, verify , approve), media , documents , trazas , effects , agent (plantillas, uso, trabajos, decisión, borrado), releases (vista previa, publicar, revertir), activations , comments , evaluacion-final , tablero , resultados y exportacion ; admin/* exige can:admin .
- **Límites de tasa**: ejecutar código 30/min, enviar 6/min, eventos 60/min, comentarios 10/min, descarga de datos propios 3/hora, inicio de sesión de la consola y agente con limitadores propios.

Lo que ve el estudiante

El aula nunca lee el diseño en edición: lee el manifiesto de la publicación vigente y lo filtra en cuatro pasos.

1. AplicadorVariantes::paraGrupo aplica la variante de su grupo y quita las tareas de otras rutas (tarea.rutas vacío = todos), con sus ayudas.
2. Disponibilidad decide qué clases están abiertas según activations (fecha general o por grupo, y si exige terminar la anterior).
3. VistaEstudiante quita la solución, los casos ocultos y el bloque diseno , y añade la etiqueta de apoyo, el papel (autoexplicación, imaginación, reto colaborativo) y la ruta.
4. La página de la clase (aula/Clase.vue) muestra Soporte, Procedimental, Tareas y Protocolo verbal en el orden del mapa de ruta; la de la tarea (aula/Tarea.vue) añade editor, ejecución y envío.

Un envío se califica en la cola (CalificarEnvio) ejecutando el código en Piston contra todos los casos, ocultos incluidos, y comparando salidas con ComparadorSalida .

Tareas programadas

`routes/console.php` : avisos de apertura de clases y revisión de operación cada 5 minutos, poda diaria de trabajos fallidos (30 días) y tokens vencidos, y limpieza de restablecimientos de contraseña cada 15 minutos.

Consola del instructor (Electron)

`apps/desktop` es una aplicación de escritorio que trabaja sin conexión: el diseño del curso vive en una base SQLite local y se sincroniza con la API de Laravel cuando hay red.

Procesos

- **Proceso main** (`src/main`): todo lo que toca red y disco. `api.ts` habla con `/api/v1`; `sesion.ts` guarda el token cifrado con `safeStorage` del sistema operativo (sin cifrado disponible, solo en memoria); `diseño/almacen.ts` es la base local (`elementos`, `conflictos`, `cursos`, `subidas`); `diseño/sincronizador.ts`, `diseño/validador.ts` (contratos), `material.ts`, `exportacion.ts` y `actualizaciones.ts` (electron-updater).
- **Preload** (`src/preload`): expone una API mínima al renderer por IPC (`ipc.ts`, `diseño/ipc.ts`).
- **Renderer** (`src/renderer`): Vue 3 + Pinia, con `sandbox`, `contextIsolation` y sin `nodeIntegration`. Vistas por curso: Diagnóstico, Estudio (diseño + asistente), Ítems, Pruebas, Materiales, Publicación, Tablero, Estudiante y Resultados.
- **Código compartido** (`src/shared`): tipos, contratos generados, reglas del diseño (`diseño.ts`) y del asistente (`agente.ts`), con pruebas en Vitest.

Sincronización del diseño

1. Cada cambio local queda pendiente con la `version` del elemento sobre la que se hizo.
2. `sincronizar` sube primero los pendientes, en lotes de 100, a `POST courses/{id}/design/sync` (con `agent_run_id` si el cambio nació de una propuesta del agente), y después baja los cambios del servidor posteriores a su último `seq`.
3. El servidor decide cada cambio con `ReglaSincronizacion`: misma versión → se aplica; versión distinta pero mismo contenido (o ambos borrados) → ya aplicado; si no → conflicto, que la consola guarda y muestra para que el instructor elija.
4. Todo cambio regresa el elemento a «borrador»: hay que verificarlo y aprobarlo otra vez antes de publicar.

Asistente del agente

En la vista Estudio, `shared/agente.ts` (`accionesPara`) ofrece lo que corresponde a lo seleccionado en el árbol (curso, clase o tarea), y `stores/agente.ts` encola el trabajo y consulta su estado. Una propuesta se revisa en `ComparadorPropuesta.vue` junto a sus validaciones: se guardan los elementos elegidos en el diseño (los bloqueados vienen desmarcados), los ítems pasan al Banco de ítems como borradores, o se elimina. «Generar tema completo» encadena las siete piezas del tema al guardar la clase (sección *Tema completo*).

Empaquetado

`electron-builder.yml` produce el instalador con *fuses* de Electron endurecidos (`runAsNode` , `NODE_OPTIONS` e inspector apagados; ASAR con validación de integridad; cookies cifradas) y auto-actualización desde un servidor genérico (`publish.url`).
`scripts/publicar-consola.ps1` firma y publica; `electron-builder.sin-firma.yml` sirve para pruebas internas. En desarrollo: `npm run dev` ; compilación sin empaquetar: `npx electron-vite build --mode development` .

Agente de IA

`services/agent` es un servicio FastAPI que recibe una solicitud de diseño ya armada por Laravel, la pasa por una plantilla, llama al modelo y entrega una propuesta validada; no guarda nada ni ve datos de estudiantes.

Endpoints

Todos, salvo `/salud` , exigen la cabecera `X-Agente-Token` con el secreto compartido `AGENTE_TOKEN` (comparación en tiempo constante; si falta o no coincide → 401).

Método y ruta	Para qué
GET <code>/salud</code>	Estado, proveedor y versión del prompt (<code>prompt</code>); sirve para confirmar que corre el código nuevo
GET <code>/v1/plantillas</code>	Plantillas disponibles (las 15 del cuadro de abajo)
POST <code>/v1/generar</code>	Genera una propuesta (<code>SolicitudGeneracion</code> → <code>ResultadoGeneracion</code>)

Método y ruta	Para qué
GET /v1/verificador/reglas , POST /v1/verificar	Reglas del verificador pedagógico y su ejecución sobre un diseño (semáforo por elemento)
POST /v1/documentos/procesar , POST /v1/embeddings	Fragmenta un PDF y calcula vectores con bge-m3 (RAG)
POST /v1/trazas/completar	Calcula los pasos de los bloques ``traza con el trazador
POST /v1/estadisticas/{pre-post, dos-grupos, ancova, correlacion}	Análisis estadístico de resultados (paso 10)

Proveedores de modelo

PROVEEDOR	Normal	Ligero	Calidad alta	Notas
local (omisión)	qwen3:4b	qwen3:4b	qwen3:8b	Ollama con salida restringida al esquema JSON, contexto 24 576, sin razonamiento, keep_alive 30 min
claude	claude-sonnet-5	claude-haiku-4-5	claude-opus-5-5	Requiere ANTHROPIC_API_KEY ; registra tokens y costo por corrida

Un error de Ollama «token repeat limit reached» (el modelo se quedó repitiendo) se trata como respuesta cortada y se reintenta; uno de conexión o de modelo inexistente es «proveedor no disponible».

Quién arma la solicitud: Laravel

EjecutarTrabajoAgente (cola agente) usa ContextoAgente para construir la solicitud: curso (título, lenguaje, nivel), el diseño actual, un resumen agregado por grupo (niveles, medias del MSLQ, proporciones y efectos sugeridos), el alcance validado por AlcancePlantilla , las indicaciones del instructor y, si hay, el material del curso. Solo viajan seudónimos y datos agregados. BuscadorMaterial elige el material: calcula el embedding de la consulta, toma hasta 6 fragmentos del mismo curso por distancia de coseno y corta en 6 000 caracteres.

Plantillas

Cada plantilla (`app/plantillas/__init__.py`) fija el paso CLT4BP, el modelo de salida (pydantic, `app/plantillas/modelos.py`), las instrucciones y la conversión a elementos del contrato.

Paso	Plantillas
1 Análisis	<code>objetivos</code>
2-4 Grupos y efectos	<code>resumen_grupo</code> , <code>preseleccion</code> , <code>diferenciacion</code> , <code>ficha_tema</code>
5 Tareas	<code>clase_tareas</code> (T1-T4 + subsolicitud T5-T8), <code>tareas_complementarias</code>
6 Soporte	<code>info_soporte</code> (conceptos), <code>ejemplo_resuelto_tema</code> , <code>mapa_glosario</code>
7 Procedimental	<code>info_procedimental</code> , <code>guion_protocolo</code>
8-10 Aplicación y evaluación	<code>plan_implementation</code> , <code>items_evaluacion</code> , <code>informe_revision</code>

El prompt de sistema (`app/conocimiento/prompt_sistema.md`) fija el marco CLT4BP, las reglas de diseño y la notación multimedia; el catálogo de 17 efectos y las reglas de preselección van en `efectos.json` y `reglas-preseleccion.json` . Todo cambio de prompt o de plantilla sube `VERSION` (hoy `sistema-v12`) y se anota en `CAMBIOS_PROMPT.md` .

El orquestador

`Orquestador.generar` hace hasta `MAX_INTENTOS` (3) intentos. En cada uno:

1. Llama al modelo con la solicitud; si la respuesta no cumple el esquema o se corta, pide corregirla (más concisa si se cortó).
2. Convierte la propuesta en elementos y la repara sin el modelo: Markdown (Mermaid, cercas, recuadros), código (cercos, ; , programas completos, `getline` para texto con espacios), nivel de apoyo según el título, código dado de autoexplicación e imaginación.
3. Calcula lo que el modelo no escribe: salidas esperadas (ejecutando la solución dos veces y con datos de más), huecos de un problema por completar, salidas y mensajes del compilador del Markdown, pasos de las trazas, rutas.
4. Valida (`app/validacion.py` , `app/multimedia.py` y el verificador `app/verificador/reglas.py`): esquema, código que compila y pasa sus casos, entradas que corresponden a lo que lee el programa, tareas distintas, desvanecimiento

por ruta, diagramas válidos y piezas completas.

5. Si quedan problemas bloqueantes, los devuelve al modelo como retroalimentación (solo la solicitud, la última respuesta y sus problemas) y repite.

Al final, lo que el modelo no pudo arreglar se degrada sin bloquear todo (trazas que no corren quedan como código, bloques de salida sin cálculo se quitan, casos que fallan se quitan si quedan dos) y lo demás queda marcado como bloqueante para el instructor. El resultado incluye elementos, notas (por ejemplo, ítems), advertencias, validaciones, intentos, uso, modelo, versión del prompt y duración.

Material del curso (RAG local, ADR 0006)

El instructor sube un PDF en Materiales; `ProcesarDocumento` (cola `agente`) lo manda al agente, que lo fragmenta y calcula vectores de 1024 dimensiones con `bge-m3`; Laravel guarda los fragmentos en `document_fragments` (pgvector, HNSW). El agente cita los fragmentos usados como [M1], [M2]... en la justificación. Cambiar de modelo de embeddings exige una migración nueva y reprocesar los documentos.

Tema completo (ADR 0007)

Un tema se genera con la forma del «Mapa de ruta CLT4BP» (el material de ejemplo del Tema 02): una clase con ocho tareas y siete piezas más, cada una como propuesta aparte que el instructor guarda o elimina.

Dónde vive cada pieza

Orden	Pieza	Plantilla	Se guarda como
1	Clase y tareas T1-T8	<code>clase_tareas (+ tareas_complementarias)</code>	<code>clase y 8 tarea con rutas</code>
2	Ficha de diseño (objetivos por ruta, evaluación, efectos, matriz Tomlinson, plan)	<code>ficha_tema</code>	<code>clase.ficha_md</code> (solo instructor)
3	Conceptos: analogía, pregunta, ¿por qué y para qué?, conceptos uno por uno, ¡cuidado!, actividad en el aula	<code>info_soporte</code>	<code>soporte modelo_mental</code>

Orden	Pieza	Plantilla	Se guarda como
4	Ejemplo resuelto: algoritmo, pseudocódigo PSeInt, código, salida, verificación	<code>ejemplo_resuelto_tema</code>	soporte <code>sap</code>
5	Mapa del tema, glosario bilingüe y fuentes	<code>mapa_glosario</code>	soporte <code>mapa_conceptual</code>
6	Tarjeta «Quiero... Escribo», guía de rutina, errores frecuentes, ejemplo isomórfico	<code>info_procedimental</code>	4 procedimentales con <code>clase_uid</code>
7	Protocolo verbal (segmentos CÓMO / POR QUÉ + programa del experto)	<code>guion_protocolo</code>	procedimental <code>protocolo_verbal</code>
8	Evaluación (recuerdo, predicción, corrección, programación)	<code>items_evaluacion</code>	ítems del Banco de ítems (borradores)

En la consola, «Generar tema completo» pide la pieza 1; al guardarla se encolan las siete restantes (`PIEZAS_DEL_TEMA`). Con una clase guardada, «Completar el tema...» las pide de nuevo y «Más tareas para...» pide T5–T8 sola.

La secuencia de tareas

T1 ejemplo resuelto y su gemelo (el código inicial es el ejemplo con su traza; solución y casos son del gemelo) · T2 por completar con pistas · T3 por completar sin pistas · T4 convencional · T5 solución libre · T6 autoexplicación · T7 imaginación · T8 reto colaborativo. Con `qwen3:4b` las ocho no caben en una respuesta: `clase_tareas` trae T1–T4 y una subsolicitud agrega T5–T8 a la misma propuesta (con un presupuesto de 600 s). El objetivo se parte en subtemas y cada tarea T1–T4 recibe uno (`reparto_de_subtemas`), para que no repitan el mismo problema.

Rutas

`asignar_rutas` reparte las tareas sin el modelo según su papel: la Ruta A (grupos de nivel básico) recibe el ejemplo, el primer por completar y la solución libre; la Ruta B (intermedio y avanzado), la autoexplicación y la imaginación; todos, el resto. Si un grupo quedara con menos de 3 tareas, todas son para todos. El instructor las cambia en el editor de la tarea, y el aula filtra con ellas.

Bloques que llena el sistema

En el Markdown del material, el modelo escribe solo el encabezado de estos bloques; el sistema ejecuta el código y pone el resto (`app/bloques_md.py` , `app/trazador.py`):

Bloque	Lo escribe el modelo	Lo pone el sistema
<code>``salida</code>	<code>entrada: ...</code> (renglones con <code>\n</code>) y <code>---</code>	La salida real del programa anterior; se calcula dos veces y con datos de más para detectar entradas incompletas
<code>``compilador</code>	Nada (bloque vacío tras un programa con error)	El mensaje real de g++, sin rutas internas de Piston
<code>``traza</code>	<code>titulo:</code> , <code>entrada:</code> , <code>---</code> y el programa	Los pasos línea por línea con variables, salida y memoria (pila, arreglos, punteros)
<code>``pseint</code>	El pseudocódigo	— (solo se dibuja)
<code>``mermaid</code>	El diagrama	Reparación de etiquetas; si falta en conceptos o mapa, un «Resumen visual» con sus conceptos

El visor de Markdown (aula `components/clt4bp/Markdown.vue` , consola `components/VistaMarkdown.vue` , lógica común en `lib/bloques.ts`) dibuja además los recuadros por su emoji (💡 ? 🎯 ⚠️ 👤 🌐 📝 🖨️ ☑️ 🐛 ↔️ 👤 🗣️) y el código anotado: una nota `//>` al final de una línea pasa a una columna junto al código.

Resultado medido con `qwen3:4b` el 2026-10-06/07: un tema tarda de 17 a 19 minutos y, en la última corrida, 7 de las 8 piezas llegaron sin bloqueantes; lo bloqueado fueron tareas o ejemplos que copiaban el problema de otra tarea.

Ejecución de código: Piston y el trazador

Todo código de estudiantes, instructores o el modelo se trata como no confiable: se compila y ejecuta solo en dos contenedores aislados, nunca en el servidor web ni en el agente.

Piston (`clt4bp-piston` , puerto 2000)

- **Lenguajes instalados:** gcc 10.2.0 (C y C++) y Python 3.12.0. Se instalan una vez con `POST /api/v2/packages ( {"language": "gcc", "version": "10.2.0"} y {"language": "python", "version": "3.12.0"} )` y quedan en el volumen `piston_packages` .
- **Quién lo llama:** Laravel (`PistonEjecutor` : diagnóstico, ejecuciones y envíos del aula, verificación de soluciones al aprobar y publicar) y el agente (`EjecutorPiston` : salidas esperadas, ejemplos, bloques de salida y del compilador). El curso `c` , `cpp` o `python` se traduce a `main.c` , `main.cpp` o `main.py` .
- **Límites por llamada (Laravel):** compilación 10 s, ejecución 3 s, 128 MB de memoria.
- **Techo en producción** (`infra/prod/piston.yml` , ninguna llamada puede pedir más): sin red, ejecución 5 s, compilación 15 s, 256 MB, 64 procesos, 32 trabajos simultáneos, salida de 64 KB.
- Requiere `privileged: true` por su sandbox (isolate) y solo escucha en `127.0.0.1` .

Trazador (`services/trazador` , puerto 2010)

Ejecuta un programa paso a paso y devuelve sus pasos reales: los pasos de una traza nunca los escribe el modelo.

- **API:** `POST /trazar` con `{"lenguaje": "c"|"cpp"|"python", "codigo", "entrada"}` → `{"pasos": [{"l": línea, "f": función, "v": [[nombre, valor]], "s": salida nueva}], "truncada", "error"}`; `GET /salud` . Solo lo llama el agente.
- **Cómo:** C y C++ se compilan con `gcc -std=c17` / `g++ -std=c++17` y se recorren con `gdb` (`traza_gdb.py`), que también arma la vista de memoria (pila de funciones, arreglos y punteros); Python con su propio rastreador (`traza_python.py`).
- **Límites:** código de 20 000 caracteres y entrada de 5 000; 300 pasos (`TRAZA_MAX_PASOS`); 4 trazas simultáneas (`TRAZA_SIMULTANEAS`); 8 s de CPU, 2 MB de salida y 64 procesos por ejecución; tiempo máximo de 20 s para compilar y 30 s para trazar.
- **Aislamiento del contenedor:** Debian slim, usuario sin privilegios (uid 10010), sistema de archivos de solo lectura con `/tmp` de 64 MB, 768 MB de memoria, 128 procesos, todas las *capabilities* quitadas y `no-new-privileges` .

- **Pruebas:** `MSYS_NO_PATHCONV=1 docker exec clt4bp-trazador python3 -m unittest discover -s /app/pruebas .`

En el aula y la consola, un bloque `` traza se dibuja como reproductor paso a paso: la línea que se ejecutará, las variables, la salida y la memoria.

Seguridad y privacidad

La plataforma guarda datos de estudiantes para una investigación: el diseño minimiza lo que sale de Laravel, exige autorización en cada ruta y deja rastro de lo que cambia.

Autenticación y autorización

- **Aula:** sesión web de Laravel con Fortify (verificación de correo, restablecimiento, verificación en dos pasos). El personal (administrador e instructor) debe activar la verificación en dos pasos: `EnsureStaffHasTwoFactor` lo redirige a Seguridad hasta hacerlo. `EnsureNotSuspended` bloquea cuentas suspendidas.
- **Consola:** `POST /api/v1/auth/login` emite un token de Sanctum con la habilidad `consola` y 7 días de vigencia (si la cuenta tiene verificación en dos pasos, la API pide el código con un `422 requires_two_factor`); el token vive solo en el proceso main, cifrado con `safeStorage` (DPAPI en Windows), y se revoca al cerrar sesión o al suspender la cuenta. Los tokens vencidos se podan a diario.
- **Agente:** secreto compartido `AGENTE_TOKEN` en la cabecera `X-Agente-Token`; el agente solo acepta llamadas de Laravel.
- **Roles** (spatie/laravel-permission): administrador, instructor y estudiante; un instructor solo ve sus cursos (`CoursePolicy`), un estudiante solo el aula de sus cursos (`acceder`), y `admin/*` exige `can:admin`.
- **Regla del proyecto:** toda ruta nueva tiene política de autorización y una prueba que verifica el 403. Las invitaciones de instructores van por enlace firmado.

Cabeceras y CSP

`PoliticaContenido` (reaplicada también en las respuestas de error desde `bootstrap/app.php`) envía una CSP con nonce por petición: `script-src 'self'` `'nonce-...'`, `object-src 'none'`, `frame-ancestors 'none'`, `base-uri` y `form-action` en `'self'`, fuentes solo de `fonts.bunny.net`; más `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy` sin cámara, micrófono, ubicación ni pagos, y `Cross-Origin-Opener-Policy: same-origin`. En local, la CSP admite el servidor de Vite.

Privacidad y datos de investigación

- **Al agente solo van seudónimos y datos agregados:** resúmenes por grupo (n, niveles, medias del MSLQ, proporciones), nunca nombres ni correos. Las exportaciones usan el seudónimo estable `E-XXXXXX`.
- **Consentimientos** (`consents`): el estudiante acepta o revoca el uso de sus datos para la investigación en «Mis datos», donde también descarga un ZIP con todos sus datos (3 descargas por hora).
- **Nada del agente se publica sin aprobación del instructor:** las propuestas se guardan como borradores, se verifican y se aprueban antes de entrar a una publicación; cada elemento conserva `autor_tipo` y `agent_run_id`.
- **Auditoría:** `audit_logs` registra acciones sensibles (por ejemplo, publicación creada o revertida, con número y huella).

Código no confiable

Todo código corre en Piston o en el trazador, con los límites de la sección anterior; la consola corre su renderer con `sandbox` y `contextIsolation`, y el instalador apaga los *fuses* que permitirían ejecutar Electron como Node.

Revisión continua

- **Dependencias:** Dependabot y un flujo semanal (`.github/workflows/auditoria.yml`) con `composer audit`, `npm audit` y `pip-audit`.
- **Escaneo ZAP:** el pasivo y casi todo el activo se completaron sin alertas; la regla de XSS por DOM no terminó por falta de memoria en el entorno local (`docs/seguridad/zap.md`).
- **Pendiente:** en el entorno local la CSP bloquea un script en línea sin nonce en todas las páginas, incluido el inicio de sesión (hash `sha256-02hx5VFZ...`); hay que identificar qué lo inserta antes del piloto.

Instalación del entorno local

El entorno de desarrollo corre en Windows 11 con Docker Desktop para los servicios y PHP, Node y Python nativos para las aplicaciones; el orden importa porque la consola y el agente dependen de la web.

Requisitos

Herramienta	Versión	Nota
Windows 11 + PowerShell 7	—	También se usa Git Bash para scripts POSIX
Docker Desktop	—	PostgreSQL, Mailpit, Piston y el trazador
PHP + Composer	8.3 o mayor	Laravel 13 lo exige; <code>php -v</code> debe decir 8.3.x
Node	24 LTS	Aula (Vite+) y consola (electron-vite)
uv	Python 3.13	Agente
Ollama (nativo)	—	Usa la GPU sin configurar Docker; con 8 GB de VRAM, <code>qwen3:4b</code>

Pasos

- Servicios** (desde la raíz): `docker compose -f infra/docker-compose.yml up -d`. Piston arranca sin lenguajes: instálalos una vez (gcc 10.2.0 y Python 3.12.0, sección *Ejecución de código*) y comprueba con `./infra/piston/probar.ps1`.
- Web y API** (`apps/web`): `composer install; npm install; cp .env.example .env; php artisan key:generate; ajusta DB_PORT=5433; php artisan migrate --seed` (requiere `ADMIN_EMAIL` y `ADMIN_PASSWORD`); `composer run dev` levanta servidor, cola por omisión, cola `agente` y Vite.
- Instrumentos** (una vez y tras cada `migrate:fresh`): `php artisan instrumentos:cargar ../../instruments/mslq.json`, igual con `cs.json` y `paas.json`. Los JSON se generan con `uv run instrumentos/generar_*.py`.
- Modelos locales**: `ollama pull qwen3:4b`, `ollama pull qwen3:8b`, `ollama pull bge-m3` (uno por comando); variables de usuario de Windows `OLLAMA_FLASH_ATTENTION=1` y `OLLAMA_MAX_LOADED_MODELS=1`, y reiniciar Ollama.
- Agente** (`services/agent`): `uv sync; cp .env.example .env` con el mismo `AGENTE_TOKEN` que `apps/web/.env`; `uv run fastapi dev app/main.py --host 127.0.0.1 --port 8100`. Comprueba `GET http://127.0.0.1:8100/salud`.
- Consola** (`apps/desktop`): `npm install; npm run dev`.
- Contratos** (si cambias un esquema): `npm run gen` y `npm run gen:py` en `packages/contracts`.

8. **Datos de prueba** (solo `APP_ENV=local`): `php artisan demo:diagnostico <id_curso> --tipo=bimodal` (`homogeneo`, `bimodal` o `disperso`) crea 30 estudiantes simulados con perfil.

Direcciones locales

Servicio	Dirección
Aula y API	http://localhost:8000 (API en <code>/api/v1</code>)
Agente	http://127.0.0.1:8100 (<code>/salud</code>)
Ollama	http://localhost:11434
PostgreSQL	127.0.0.1:5433, base y usuario <code>clt4bp</code>
Piston	http://127.0.0.1:2000/api/v2
Trazador	http://127.0.0.1:2010
Correo (Mailpit)	http://localhost:8025

Una generación larga (un tema completo tarda unos 17 minutos) conviene lanzarla desacoplada de la terminal (por ejemplo, con `Start-Process` de PowerShell) para que no muera si se cierra la sesión.

Variables de configuración

La configuración vive en dos archivos `.env` (plantillas en `apps/web/.env.example` y `services/agent/.env.example`; en producción, `infra/prod/web.env.ejemplo` y `agente.env.ejemplo`); los secretos nunca se versionan.

Aula y API (`apps/web/.env`)

Variable	Valor local	Para qué
<code>APP_ENV</code> , <code>APP_URL</code> , <code>APP_LOCALE</code>	<code>local</code> , <code>http://localhost:8000</code> , <code>es</code>	Entorno, URL pública, idioma
<code>DB_CONNECTION</code> , <code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_DATABASE</code> , <code>DB_USERNAME</code> , <code>DB_PASSWORD</code>	<code>pgsql</code> , <code>127.0.0.1</code> , <code>5433</code> , <code>clt4bp</code> , <code>clt4bp</code> , (secreto)	Base de datos del contenedor
<code>QUEUE_CONNECTION</code>	<code>database</code>	Colas en la base

Variable	Valor local	Para qué
DB_QUEUE_RETRY_AFTER	1000	Debe superar AGENTE_TIMEOUT + 60, o la cola relanza una generación en curso
SESSION_LIFETIME	120	Minutos de sesión web
ADMIN_NAME , ADMIN_EMAIL , ADMIN_PASSWORD	—	Administrador inicial que crea el seeder; borrar la contraseña después
PISTON_URL	http://127.0.0.1:2000/api/v2	Ejecución de código
AGENTE_URL , AGENTE_TOKEN	http://127.0.0.1:8100 , (secreto)	Agente; el token es el mismo en ambos .env
AGENTE_TIMEOUT	900 con modelo local, 280 con Claude	Segundos que puede tardar una generación
MAIL_FROM_ADDRESS , MAIL_FROM_NAME	no-responder@clt4bp.local	Remitente de avisos
CLT4BP_VERSION_PRIVACIDAD , CLT4BP_VERSION_INVESTIGACION	2026-09	Versión de los avisos que acepta el estudiante
CLT4BP_MEDIOS_MAX_KB , CLT4BP_MATERIAL_MAX_KB	512000, 51200	Tamaño máximo de medios y de PDF del material
CLT4BP_AGENTE_LIMITE_USD , CLT4BP_ALERTA_GASTO_DIARIO_USD	20, 5	Límite mensual de gasto con Claude y alerta diaria

Variable	Valor local	Para qué
CLT4BP_ALERTAS_CORREO , CLT4BP_PING_REVISION	—	Destinatario de alertas y URL «sigo vivo» de la revisión de operación
CLT4BP_CONTACTO_PRIVACIDAD	—	Contacto que muestran los avisos legales
SENTRY_LARAVEL_DSN	vacío	Monitoreo de errores en producción

Agente (services/agent/.env)

Variable	Omisión	Para qué
AGENTE_TOKEN	(secreto)	Igual al de Laravel
PROVEEDOR	local	local (Ollama) o claude
PISTON_URL , TRAZADOR_URL	http://localhost:2000 , http://localhost:2010	Ejecución y trazas
OLLAMA_URL	http://localhost:11434	Ollama
MODELO_LOCAL_NORMAL , MODELO_LOCAL_LIGERO , MODELO_LOCAL_ALTO	qwen3:4b , qwen3:4b , qwen3:8b	Modelos locales; con 12 GB de VRAM o más, qwen3:8b como normal
OLLAMA_NUM_CTX	24576	Contexto; 32k ya no cabe en 8 GB con qwen3:4b
OLLAMA_PENSAR	no	Razonamiento (no , si , omitir); si duplica el tiempo
OLLAMA_TIMEOUT , OLLAMA_KEEP_ALIVE	900, 30m	Espera máxima y cuánto sigue cargado el modelo

Variable	Omisión	Para qué
MODELO_EMBEDDINGS	bge-m3	Embeddings (1024 dimensiones, fijas en la migración)
ANTHROPIC_API_KEY , MODELO_NORMAL , MODELO_LIGERO , MODELO_ALTO	—, claude-sonnet-5 , claude-haiku-4-5 , claude-opus-5-5	Solo con PROVEEDOR=claude
MAX_INTENTOS	3	Intentos por propuesta
TAREAS_COMPLEMENTARIAS	true	Pedir T5–T8 dentro de la propuesta de la clase
CLASE_COMPLETA	false	Pedir soporte y ayudas dentro de la propuesta de la clase (lo reemplaza la cadena del tema)
PRESUPUESTO_CLASE_S	600	Segundos tras los cuales ya no se piden subsolicitudes; menor que AGENTE_TIMEOUT
SENTRY_DSN	vacío	Monitoreo de errores

Variables de usuario de Windows para Ollama: OLLAMA_FLASH_ATTENTION=1 y OLLAMA_MAX_LOADED_MODELS=1 . No usar OLLAMA_KV_CACHE_TYPE=q8_0 : en este equipo vuelve lentísima la lectura del prompt.

Producción y operación

Producción corre en un solo servidor Ubuntu Server 24.04 LTS (4 vCPU, 8 GB de RAM, 100 GB de disco) con todo lo instalado desde infra/prod ; el procedimiento completo está en docs/despliegue.md y la rutina en docs/operacion.md .

Piezas del servidor

Archivo en infra/prod	Destino	Para qué
preparar-servidor.sh	Se corre una vez	Instala y configura todo lo de esta tabla

Archivo en infra/prod	Destino	Para qué
nginx/clt4bp.conf	/etc/nginx/sites-available/clt4bp	Sitio web, descargas de la consola y HSTS
php/clt4bp.conf	/etc/php/8.5/fpm/pool.d/	Pool de PHP-FPM como usuario deploy
supervisor/clt4bp.conf	/etc/supervisor/conf.d/	Trabajadores de las colas default y agente
systemd/clt4bp-agente.service	/etc/systemd/system/	Agente en 127.0.0.1:8100
piston.yml , trazador.yml	/opt/clt4bp-piston/compose.yml y equivalente	Piston y trazador con límites de producción
desplegar.sh , regresar.sh	/srv/clt4bp/	Desplegar una etiqueta y volver a la anterior
respaldar.sh , probar-restauracion.sh	/srv/clt4bp/ , clt4bp-probar-restauracion	Respaldo cifrado diario y su prueba
web.env.ejemplo , agente.env.ejemplo , restic.env.ejemplo	/srv/clt4bp/shared/ , /etc/clt4bp/	Configuración (sin «ejemplo»)

/srv/clt4bp contiene repo/ (espejo del repositorio), releases/ (una carpeta por despliegue; se conservan cinco), current (enlace a la vigente) y shared/ (.env , archivos subidos, bitácoras y descargas de la consola).

Desplegar

1. Fusionar en main con la integración continua en verde.
2. Etiquetar y publicar la etiqueta: `git tag v1.0.1; git push --tags`.
3. `ssh -t <usuario>@<servidor> sudo -iu deploy /srv/clt4bp/desplegar.sh v1.0.1`.
4. Si sale mal: `regresar.sh` , y después investigar.

Mientras haya estudiantes, las migraciones solo agregan (tablas, columnas que admiten nulo, índices); quitar o renombrar va en dos despliegues para que `regresar.sh` siempre funcione. Un cambio delicado va con `artisan down --refresh=15` fuera del horario de

clase.

Respaldos

`respaldar.sh` vuelca PostgreSQL y respalda el volcado, los archivos subidos y las configuraciones con restic, cifrados, en un repositorio fuera del servidor (SFTP institucional o almacenamiento compatible con S3). Retención: 14 diarios, 8 semanales y 12 mensuales; cada corrida verifica al azar el 5 % de los datos. Una vez al mes se prueba la restauración (`clt4bp-probar-restauracion`).

Monitoreo

- **Dentro del servidor:** `operacion:revisar` (cada 5 minutos, `App\Domain\Operacion\Diagnostico`) avisa por correo si el agente o Piston no responden, si una cola tiene trabajos viejos, si hubo trabajos fallidos en la última hora, si queda poco disco o si el gasto del agente del día pasa el umbral.
- **Desde fuera:** un servicio externo con tres vigilancias: `/up` debe responder 200 cada 5 minutos; un latido de la revisión (`CLT4BP_PING_REVISION` , cada 5 minutos con 10 de margen) y uno del respaldo (`PING_RESPALDO` , diario con 2 horas de margen).
- **Errores:** Sentry en Laravel y en el agente.
- **Gasto del agente** (con Claude): cuota mensual por instructor, alerta diaria y el límite del espacio de trabajo en la Claude Console.

Rutina

Cuándo	Qué revisar
Cada día de clase	Correos de alerta; alertas pedagógicas del tablero en la consola
Cada semana	Sentry, <code>sudo supervisorctl status</code> , <code>df -h</code> , pull requests de Dependabot (en el piloto, solo los de seguridad)
Cada mes	Prueba de restauración, gasto del agente, <code>certbot renew --dry-run</code> , cuentas del personal
Al cerrar el piloto	Exportación final, respaldo etiquetado <code>cierre</code> y borrado o anonimización según el protocolo

Pruebas y calidad

Cada componente tiene su suite automatizada, y la integración continua (`.github/workflows/ci.yml`) corre cuatro trabajos en cada push: contratos, web, consola y agente. Estado al 2026-10-07: todas en verde.

Suite	Comando	Alcance	Pruebas
Aula y API	<code>cd apps/web; php artisan test</code>	Dominio (unitarias, sin base) y feature (rutas, 403, aula, agente, publicación); usa una base de prueba con pgvector y un ejecutor de código falso	199
Agente	<code>cd services/agent; uv run pytest</code>	Plantillas, orquestador, etapas, bloques calculados, multimedia, verificador, proveedores (Ollama y Claude simulados)	135
Consola	<code>cd apps/desktop; npm test y npm run typecheck</code>	Reglas del diseño, sincronización, asistente, visor de bloques (Vitest) + tipos	27
Tipos del aula	<code>cd apps/web; npx vue-tsc --noEmit</code>	Páginas y componentes Vue	—
Contratos	<code>cd packages/contracts; npm run check</code>	Ejemplos válidos contra los esquemas; en CI, también que las copias generadas estén al día	—
Trazador	<code>MSYS_NO_PATHCONV=1 docker exec clt4bp-trazador python3 -m unittest discover -s /app/pruebas</code>	Pasos de C, C++ y Python	—

Casos especiales: `PistonRealTest` usa el Piston real y se omite solo si no está disponible; el resto usa un ejecutor falso.

Pruebas contra un servidor real

- **Extremo a extremo** (`e2e/` , Playwright): un estudiante ejecuta y envía código en una tarea; el aula envía su política de contenido; el verificador responde con su semáforo; sin token la API responde 401. Variables `E2E_URL` , `E2E_CURSO` , `E2E_TAREA` ,

`E2E_PASSWORD` , `E2E_TOKEN` .

- **Carga** (`carga/aula.js` , k6): una clase completa sobre la misma tarea, de 0 a 100 usuarios virtuales en 7 minutos, ejecutando y enviando código.
- **Seguridad**: escaneo ZAP pasivo y activo (`docs/seguridad/zap.md`).
- **Usabilidad**: calculadora del SUS (`docs/usabilidad/sus.py`); el estudio con participantes está pendiente.
- **Estadística**: `docs/validacion/estadisticas.R` replica en R los análisis del agente para compararlos.

Calidad del agente

- **Regresión**: `services/agent/regresion` tiene 20 casos; `uv run python -m regresion.correr` genera un `resumen.csv` por versión del prompt (`--proveedor claude` para compararlo con Claude). Pendiente: correrlos con `sistema-v12` .
- **Rúbrica experta** (`regresion/rubrica.md`): alineación con los objetivos, fidelidad CLT4BP, desvanecimiento y variabilidad, autenticidad y ARCS, adecuación al grupo, de 1 a 4.
- **Corrida real del tema completo**: se generaron las ocho piezas con el modelo local y se publicaron en un curso de demostración para revisarlo en el aula con un estudiante de cada ruta; los scripts de esa revisión no están en el repositorio (sección *Tema completo*).

Formato

Aula: `npx vp fmt` (no Prettier); consola: Prettier y ESLint; PHP: `vendor/bin/pint` sobre los archivos cambiados. Los archivos GENERADO están excluidos del formateador.

Solución de problemas

La mayoría de las fallas vistas en desarrollo vienen del entorno (Docker, colas, variables, GPU), no del código: revísalo primero.

Síntoma	Causa probable	Arreglo
<code>SQLSTATE[08006] ... port 5433 failed: Connection refused</code>	Docker Desktop cerrado (pasa tras reiniciar el equipo) o el contenedor <code>clt4bp-db</code> detenido	Abrir Docker Desktop, <code>docker compose -f infra/docker-compose.yml up -d</code> ; comprobar <code>DB_PORT=5433</code>

Síntoma	Causa probable	Arreglo
<code>php -v</code> no dice 8.3	Otro PHP antes en el <code>PATH</code>	Usar el PHP 8.3 de winget; el viejo de <code>C:\php</code> está renombrado a <code>php8.2-viejo.exe</code>
Propuestas o material «en cola» para siempre	No corre el trabajador de la cola agente	<code>composer run dev</code> lo incluye; si no, <code>php artisan queue:work --queue=agente</code>
Una generación se corta o se repite sola	<code>AGENTE_TIMEOUT</code> o <code>DB_QUEUE_RETRY_AFTER</code> cortos para el modelo local	<code>AGENTE_TIMEOUT=900</code> y <code>DB_QUEUE_RETRY_AFTER=1000</code>
401 Token del agente inválido	<code>AGENTE_TOKEN</code> distinto en los dos <code>.env</code>	Igualarlos y reiniciar el agente y la cola
El agente responde con el código anterior	En Windows, la recarga de <code>fastapi dev</code> se cuelga («Reloading...») y sigue el proceso viejo	Reiniciar el agente y comprobar que <code>GET /salud</code> diga la <code>VERSION</code> actual en <code>prompt</code>
Generación de 5-6 tokens/s	El modelo no cabe en la GPU (<code>qwen3:8b</code> en 8 GB, otras aplicaciones usando VRAM o <code>OLLAMA_KV_CACHE_TYPE=q8_0</code>)	<code>ollama ps</code> debe decir 100 % GPU; usar <code>qwen3:4b</code> ; quitar <code>OLLAMA_KV_CACHE_TYPE</code>
Ollama: «token repeat limit reached»	El modelo se quedó repitiendo	El agente reintenta solo; si persiste, regenerar la pieza
La consola abre con <code>Cannot read properties of undefined (reading 'isPackaged')</code>	<code>ELECTRON_RUN_AS_NODE</code> definida en la terminal	Quitar la variable
«Ejecutar» falla para todos	Piston caído o sin lenguajes instalados	<code>docker ps</code> ; instalar gcc 10.2.0 y Python 3.12.0; <code>./infra/piston/probar.ps1</code>
Una traza queda sin pasos	El trazador no responde o el programa no compila	<code>GET http://127.0.0.1:2010/salud</code> ; en la consola, «Calcular pasos»

Síntoma	Causa probable	Arreglo
Un material queda en «Error»	PDF escaneado sin texto, o falta <code>bge-m3</code>	Pasarlo por OCR; <code>ollama pull bge-m3</code> ; eliminarlo y subirlo de nuevo
Envíos en «Calificando...»	Trabajador de la cola <code>default</code> detenido	<code>php artisan queue:work</code> ; en producción, <code>sudo supervisorctl restart c1t4bp-cola:*</code>
Al programar fechas: «la clase no está en la publicación vigente»	La clase aún no se publica	Publicar primero; el plan solo incluye clases de la publicación vigente
Una propuesta llega con bloqueantes	El modelo no los resolvió en 3 intentos	Revisarla en el comparador (lo bloqueado viene desmarcado), editar o regenerar la pieza; con <code>PROVEEDOR=claude</code> hay menos
Una generación larga muere a medias	Se cerró la terminal o la sesión que la lanzó	Lanzarla desacoplada (<code>Start-Process</code>) o desde la consola, que la deja en la cola

En producción, la tabla «Síntoma → primer paso» de `docs/operacion.md` cubre además web caída, disco lleno, una versión que rompió algo y sospecha de acceso indebido.

Decisiones de arquitectura y pendientes

Las decisiones de fondo se registran como ADR en `docs/decisiones` ; antes de cambiar algo de lo que fijan, se escribe un ADR nuevo.

ADR	Decisión
0001 Stack tecnológico	Laravel + Inertia/Vue, Electron + Vue, FastAPI, PostgreSQL + pgvector y Piston; sin Redis ni MinIO en la v1
0002 Sesión de la consola	Token de Sanctum con habilidad <code>consola</code> y 7 días de vigencia; inicio en dos pasos; el token vive solo en el proceso main, cifrado con <code>safeStorage</code>

ADR	Decisión
0003 Reglas del perfil	Conocimiento previo = 0.5 teórico + 0.5 práctico; niveles con cortes en 40 y 70; grupo homogéneo si $CV \leq 0.25$ y ≥ 70 % en el nivel modal; recomendación confiable con $n \geq 10$; MSLQ de 73 ítems y 13 subescalas
0004 Umbrales del verificador	Valores de partida de las reglas pedagógicas (por ejemplo, 0.8 de similitud para «variabilidad»)
0005 lodash-es	Forzar <code>lodash-es</code> $\geq 4.18.1$ por <i>override</i> en aula y consola
0006 Agente local y RAG	Ollama por omisión (<code>qwen3:4b</code> en 8 GB de VRAM), Claude como opción; el agente fragmenta y calcula embeddings, Laravel es dueño del índice y elige el material
0007 Forma del material	Tema completo con la forma del «Mapa de ruta CLT4BP»: piezas en cadena, procedimental del tema, tareas por ruta, salidas y mensajes del compilador calculados

La bitácora del prompt del agente está en `services/agent/CAMBIOS_PROMPT.md` y la del proyecto en `CHANGELOG.md` y `docs/piloto/bitacora.md`.

Pendientes

- ☐ Correr los 20 casos de regresión con `sistema-v12` y compararlos con Claude (requiere `ANTHROPIC_API_KEY`).
- ☐ Recorrido manual en la consola: subir un PDF en Materiales, generar un tema completo, guardar y publicar.
- ☐ Revisión visual del tema completo en la consola (en el aula ya se hizo con estudiantes de las rutas A y B).
- ☐ Identificar el script en línea que bloquea la CSP en todas las páginas.
- ☐ Exportar el tema completo a PDF con la forma del mapa de ruta.
- ☐ Actualizar el manual del instructor (generar tema completo, rutas, Banco de ítems).
- ☐ Recorridos manuales pendientes de etapas previas (6.5 y Etapas 3 a 5), cubiertos en parte por Playwright.
- ☐ Piloto `v1.0.0`: puesta a cero, aprobación del comité de ética, servidor de producción real y estudio de usabilidad SUS con participantes (`docs/piloto.md`).
- ☐ Crear las etiquetas v0.1.0 a v0.5.0 (existe v0.7.0).